Sprint 4 · Python para IA

¿cómo automatizamos lo que hemos hecho a mano?

FECHAS
27–29/10

HORAS
3 h de teoría (T) · 5 h de práctica (P) · 2 h de proyecto (PBL) = 10 h

Qué aprenderás

1. A trabajar en Colab o Jupyter y a escribir Python básico: variables, tipos, decisiones y bucles.
2. A crear **funciones** y **módulos** reutilizables, con documentación y pruebas.
3. A manejar **listas, diccionarios, conjuntos** y comprensiones para representar datos reales.
4. A leer y escribir **CSV y JSON** con la biblioteca estándar y a controlar errores con excepciones.
5. A organizar tu código con clases, buenas prácticas y **Git básico**, y a entregar una herramienta con pruebas.

Requisitos previos

- Cuenta de Google para usar Colab (o Python 3.11 y Jupyter/VS Code en tu equipo).
- **No hace falta saber programar.** Si vienes de Java, en los cuadernos hay comparativas con Python.
- Los datos (`reservas_turisdata.csv`) los descargan los cuadernos solos; para el reto los tendrás también en el repositorio.
- Recuerda: los datos son **sintéticos** (no son estadísticas reales del turismo canario).

Prácticas (5 h)

Ejecuta los cuadernos en orden. Cada ejercicio se **autocorrige**: si la celda de comprobación falla, lee el mensaje, corrige y vuelve a ejecutar.

Cuaderno	Tema	Tiempo
`S04_01_entorno_sintaxis`	Entorno, variables, tipos, operadores, `if`, `for`, `while`	60 min
`S04_02_funciones_modulos`	Funciones, parámetros, `*args`, `lambda`, módulos	60 min
`S04_03_estructuras_datos`	Listas, tuplas, diccionarios, conjuntos, comprensiones	60 min
`S04_04_ficheros_excepciones`	CSV y JSON, `try/except`, lector robusto	60 min
`S04_05_poo_buenas_practicas`	Clases, herencia, buenas prácticas, Git básico	60 min

RETO Reto PBL · «Herramientas internas» (2 h)

Contexto del cliente

El equipo de reservas de *TurisData Canarias* calcula cada semana sus cifras a mano en una hoja de cálculo: es lento y cada persona lo hace un poco distinto. Os pide una **herramienta interna**: un script que lea el CSV de reservas, calcule tres KPIs, avise con claridad si el fichero está mal y deje los resultados en un JSON que otros sistemas puedan leer.

Qué tenéis que entregar

Un **repositorio Git** con:

- `kpis.py`: el script (funciones + `main`), solo con la biblioteca estándar (`csv`, `json`…).
- `test_kpis.py`: las pruebas con `assert` (os damos una base; podéis añadir las vuestras).
- `README.md`: cómo se ejecuta, cómo se prueba, definición de cada KPI y decisiones (plantilla en `reto_herramientas/README_PLANTILLA.md`).
- Un `salida.json` generado con el CSV real.
- *(Opcional)* `kpis_pandas.py`: la misma lógica con pandas.

KPIs: ocupación aproximada (*proxy*), ingreso medio por reserva y tasa de cancelación por canal. Las definiciones exactas están en los docstrings de la plantilla.

Restricciones

- **Tiempo:** 2 h dentro del sprint.

- Versión principal solo con la **biblioteca estándar**; pandas solo en la parte opcional.
- El script **no puede romperse** con un fichero inexistente, con columnas que faltan o con filas defectuosas: debe informar.
- Sin datos personales reales ni claves en el repositorio.

Criterios de evaluación (rúbrica del curso, 0–4 en cada criterio)

Criterio	Qué se mirará en este reto
Corrección técnica	Las pruebas pasan, los KPIs coinciden con los esperados, los errores están controlados.
Reproducibilidad	Otra persona clona el repositorio, sigue el README y obtiene el mismo JSON.
Análisis y comunicación al cliente	El README y el JSON se entienden sin ser programador; cada KPI está definido y sus límites explicados (por qué la ocupación es solo un proxy).
IA responsable	Se explica qué parte se ha hecho con ayuda de IA y qué se ha revisado; se mencionan los límites de datos sintéticos y del proxy.

Pasos sugeridos

- **Día 1 (mitad del tiempo):** copiad la carpeta `reto_herramientas/`, ejecutad `python test_kpis.py`, implementad `leer_reservas` y las funciones de KPI una a una hasta que pasen las pruebas.
- **Día 2:** `exportar_json` y `main`, ejecutad con el CSV real, escribid el README y haced los *commits* (`git init`, `git add`, `git commit`). Opcional: versión pandas.

Qué se enseña en la review

Una demo de 3–5 minutos: ejecutar el script con el CSV real, mostrar el JSON, provocar un error a propósito (fichero inexistente) y enseñar el mensaje, y mostrar las pruebas en verde. Explicad una decisión de diseño (por ejemplo, cómo definisteis la ocupación).

Definition of Done

- [ ] `python test_kpis.py` termina con todas las pruebas superadas.
- [ ] `python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los tres KPIs y la sección `calidad`.
- [ ] El script informa con un mensaje claro si falta el fichero o una columna, y descarta y cuenta las filas defectuosas.
- [ ] Todas las funciones tienen docstring y siguen PEP 8 (nombres, sangría).
- [ ] El README explica cómo ejecutar, cómo probar y las definiciones de los KPIs.
- [ ] El repositorio tiene al menos 3 *commits* con mensajes claros y no contiene claves ni datos personales.
- [ ] Alguien del grupo ha revisado el código con la rúbrica.

Recursos

- Tutorial oficial de Python (en español): https://docs.python.org/es/3/tutorial/
- Módulo `csv`: https://docs.python.org/es/3/library/csv.html · Módulo `json`: https://docs.python.org/es/3/library/json.html
- PEP 8, guía de estilo (resumen): https://peps.python.org/pep-0008/
- Git, guía básica: https://git-scm.com/book/es/v2
- Google Colab, primeros pasos: https://colab.research.google.com/